Live Interactive Visual Teaching

QWEEPS / TECHNICAL REPORT 2.1

Rayven Pahignalo | ORCID: 0009-0003-2325-7968

8 November 2026 | Architecture walkthrough preview and release snapshot

Software DOI: 10.5281/zenodo.23791785

An explanation you can explore

QWEEPS turns a question or supplied document into a connected visual lesson. A Python teaching harness plans reasoning beats, directs diagrams and animation, checks content, and reviews rendered frames. A React player combines seekable visuals, narration, contextual questions, focused lesson branches, and reinforcement quizzes.

The default route uses live SVG/GSAP browser scenes through a Hyperframes adapter. An optional desktop hybrid route interleaves those scenes with generated ManimCE-style source compiled into a live ManimGL runtime. Neither live route requires an MP4. A separate Android wrapper bundles the browser studio and Python backend in one APK.

This revision adds the experimental architecture and code-tracing workspace to the visual-lesson report. Its procedural compilers, nested inspection, packet timeline, saved walkthroughs and OpenCode Go explanations are implemented, but the architecture feature remains incomplete. Voice explanation comparable to the visual lessons is planned and is not integrated into this workspace.

> Evidence boundary. This is an implementation report. Automated tests, selected rendered examples, and small provider probes establish specific engineering behavior; they do not establish learning gains, uniform generation quality, or a universal model ranking.

Revision scope. Version 2.1 supersedes report 2.0 dated 6 November 2026. It describes the accompanying source snapshot, including the OpenCode Go switch and main-menu logo link. The release archive records its base commit, working-tree changes and per-file hashes; its manifest identifies the exact deposited bytes even when changes are not yet committed. Report versions are independent of the application package version. Historical measurements retain their original scope.

Reading guide. Pages 2-4 cover visual lessons; 5-6 describe the incomplete architecture workspace and planned voice work; 7-10 retain narration, quality and model evidence; 11-14 cover platforms, verification, limitations and reproduction.

01 / LEARNER EXPERIENCE

A compact workspace for learning

The current interface uses a restrained pixel-art identity: a Pixel Q logo, dark green surfaces, bordered controls, gold highlights, and a subtly animated background. Display typography carries the theme while explanatory text needs readable sizing and contrast. The interface has been revised in response to oversized panels, weak button contrast, excessive text glow, and wasted lesson space.

Create, revisit, practice

Area	Current behavior
Create	Enter a learning request; select saved source material or attach a document; choose quiz and optional native-animation generation where available.
Library	Reopen saved lessons, their quizzes, or saved architecture walkthroughs. The experimental Fast chapter stream button and page have been removed from the learner interface.
Lesson	Watch a connected explanation, pause or seek, change chapters, inspect a visible object or region, and ask a question in context.
Practice	Answer reinforcement cards, inspect feedback, revisit the corresponding beat, and review set-level or whole-quiz results.

Keep the stage available

Inside a lesson, the main navigation is tucked into a top-left menu. Redundant page labels and the separate object-instruction/action bar have been removed. Captions appear over the stage on a transparent background, and the native authoring harness reserves a lower region so generated text does not compete with those captions.

Mobile layouts use the available viewport in portrait and landscape. Chapter selection uses a native select, and secondary quiz controls expand on demand. Settings and source entry allow scrolling where their forms need it. These are responsive layout choices, not a promise that arbitrary generated content will fit every screen without clipping.

Preparation is part of the experience

Architecture walkthroughs have a separate compact debugger-style workspace with the Pixel Q identity, code context and nested inspection. Its logo returns to the main menu. This workspace is distinct from the visual-lesson player described above.

The preparation screen now uses the same visual identity as the studio. It reports the active generation stage and supports cancellation. A long-running model request, speech synthesis, or visual review remains real work; the interface should communicate that state without implying that the lesson is already approved.

> Scope of removal. The learner-facing chapter-stream page is retired. Internal streaming endpoints remain for existing programmatic consumers; removing a button is not removal of the underlying APIs.

02 / TEACHING AND REINFORCEMENT

Show the relationship, then test it

A beat is a step that answers a learner question through visible evidence. A moving label is not automatically an explanation. New plans declare a presentation method, its rationale, visual evidence, and text mode. Those declarations make the intended operation inspectable; rendered review must still determine whether it occurred. [1]

Learning question	Appropriate visible operation
Why does a relation hold?	Construct geometry or transform matching quantities; retain enough labels to identify what changed.
How does a mechanism work?	Track a concrete input through changing state, signal flow, or a moving part.
Who acted, where, and why?	Use recognizable sourced geography, named roles, ordered consequences, and useful descriptions.
How do cases differ?	Align examples on an explicit comparison criterion; keep the distinction visible.

One example across the explanation

Stable identities, colors, units, values, and source constraints connect the beats. A spoken bridge explains why the next question follows from the previous result. Native generation receives the shared example and browser-layout context so switching renderer does not license an unrelated example or incompatible values.

Labels mode favors geometry and concise identification. Descriptive mode retains definitions and explanations when they are needed to understand a history, process, or infographic. A caption-free diagnostic tests whether a relationship is visible; it is not a ban on useful explanatory text.

Reinforcement quiz

Practice includes multiple-choice cards and supported typed-recall modes, immediate feedback, streak and mastery indicators, and set-level review. Missed concepts return through a spaced-recall queue, including the three-card retry behavior demonstrated in the P vs NP example. Learners can inspect a related lesson beat and return to practice.

The quiz header consolidates mode, round, streak, set selection, and pixel-segment progress. Redundant instructional bars have been removed. On mobile, all four answer choices remain together: one column in portrait and two columns beside the prompt in landscape. Answer selection submits directly; feedback and set transitions remain distinct interaction states.

> Interpretation. Reaching the quiz's internal mastery state means satisfying its answer logic. It does not demonstrate long-term retention, transfer, or a measured educational advantage over another teaching system.

03 / IMPLEMENTED ARCHITECTURE

One harness, two live renderers

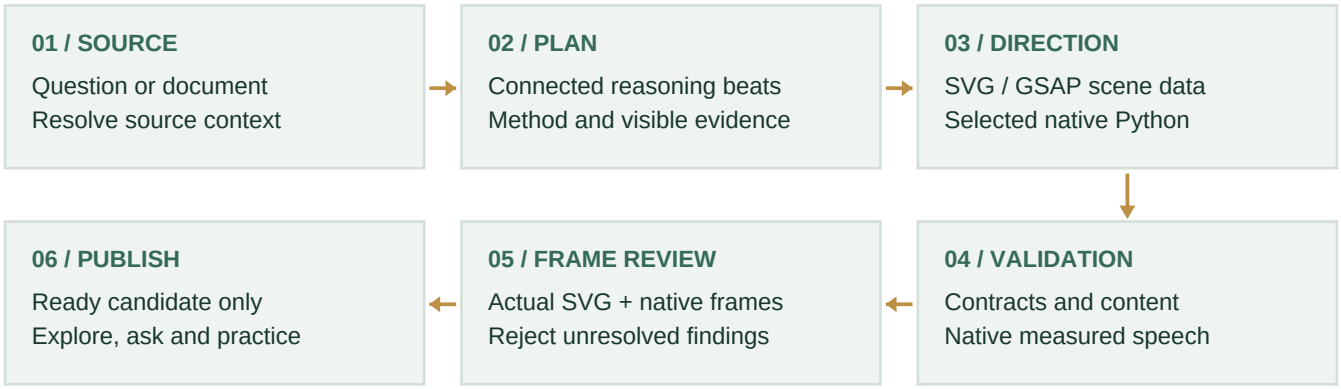


Figure 1. Simplified current preparation flow. Native speech is prepared before its final frame review so captures can use the measured audio clock. Browser preparation retains its own review scheduling. Publication waits for required review and configured speech preparation.

The Python lesson store owns source resolution, storyboard planning, direction, validation, checkpoints, review, and publication. React owns the player and browser capture. External models generate and inspect candidate content; their output remains untrusted data, subject to the scene contracts. [1]

Component	Responsibility and boundary
Browser scenes	SVG geometry and text, seekable GSAP animation, and the Hyperframes adapter. Review images are evidence, not the playback medium.
Native scenes	ManimCE-style Python authored for a supported compatibility layer, compiled and recorded for live ManimGL frames. This is not unrestricted CE execution or video playback.
Hybrid assembly	Retains browser chapters and maps selected native chapters into the shared lesson timeline. Generates Python only for the native subset.
Publication	Keeps a candidate separate until required gates pass; replaces the published manifest atomically when ready. A failed native candidate preserves an existing published lesson.

Explore without losing your place

Question context can include the current scene, timestamp, selected objects, camera, region, and experiment state. A focused visual branch retains a parent checkpoint. Returning restores the parent lesson context and audio position. Generated choices are bounded explanatory cases, not a general simulation engine.

Renderer identity matters

The active desktop native renderer is ManimGL. ManimCE-style authoring, CE metrics or export, and ManimGL playback are related but separate paths. The Android APK runs browser scenes locally; it does not bundle the desktop OpenGL renderer. A functioning renderer is necessary, but does not establish factual or teaching quality.

04 / ARCHITECTURE WORKSPACE PREVIEW

Follow data through a system

The complete supplied architecture prototype, update 3, now runs inside QWEEPS. Its compilers, ELK layout, SVG/React canvas, timeline, sidebar and nested lessons replace the earlier architecture UI. This is an experimental workspace, not a completed extension of the visual-lesson harness. [1,8]

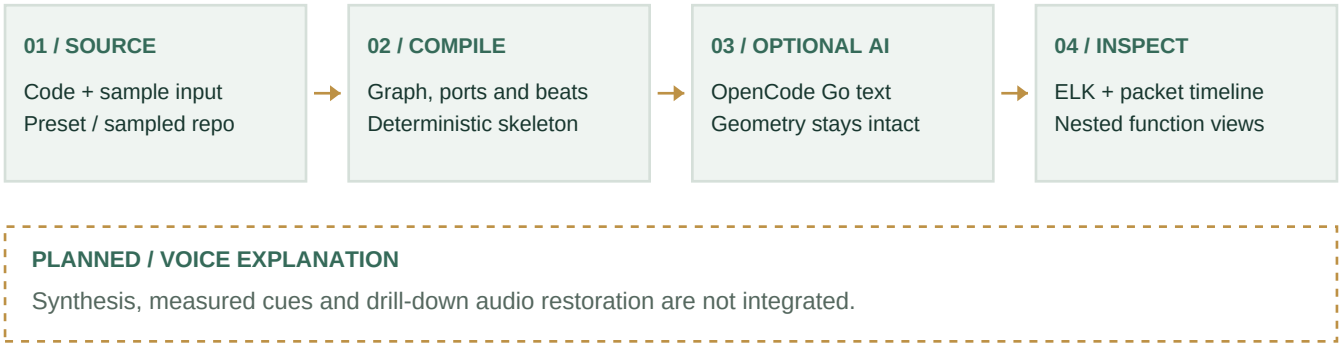


Figure 2. Architecture preview pipeline. Compilation, inspection and saving work without AI credentials. Optional AI fills explanation slots after compilation. The separate planned voice layer is not connected to the current playback clock.

Implemented layer	Current behavior
Compile	Accept source code and sample input, choose a supplied preset, or request a sampled public GitHub scan. Construct graph and beat data procedurally.
Inspect	Move from the full-system blueprint into function logic and a deeper flowchart. Show input/output cards, worked-example values and reactive previews.
Play and return	Animate packets and preview state with a seekable clock. Support pause, rewind, focused single-beat playback and return to saved parent context.
Save and reopen	Store complete bundles locally and list them in the QWEEPS library. Engine revisions trigger recompilation of older saved source/input.
Explain	Use OpenCode Go for explanatory gap filling and contextual tutor replies. Preserve the prototype's system instructions, user prompts and response schemas.

Shared provider, separate teaching route

The inspected architecture route uses the shared Go key and configured visual model, currently Muse Spark 1.3 Contributor. Credentials stay in the local backends. AI changes text slots, not graph identities, geometry or example values. The route does not run the visual-lesson storyboard, frame-review or narration publication gates; those results must not be assumed for an architecture bundle.

05 / ARCHITECTURE STATUS AND NEXT WORK

Useful preview, incomplete system

A convincing blueprint is not proof that arbitrary source code has been understood or executed. The supplied default repository walkthrough contains an authored teaching pipeline and illustrative values. The repository compiler is template-based; it does not yet establish an exhaustive, source-verified call graph for any repository. [8]

Boundary	What this snapshot establishes
Source inspection	The GitHub scanner samples up to fourteen supported files and up to 1,200 lines per file, using pattern-based import/definition extraction. It is not whole-program analysis.
Worked examples	Values, packets and node states support explanation. Displayed source is not executed in a sandbox, and fixture values are not runtime traces.
General design	Presets and supported code patterns compile. Arbitrary natural-language system design and unfamiliar repositories still require broader compiler and authoring work.
Quality evidence	Schema checks, upstream fidelity and selected browser interactions are tested. There is no architecture learning-outcome study or equivalent visual-lesson approval gate.
Platform	This workspace requires the local Node/Express backend. The current Android APK does not bundle Node; architecture API parity is not established.

Planned: voice explanation like the visual lessons

The intended next step is spoken explanation of the current architecture beat while the learner watches inputs, packets, function transformations and outputs. Existing visual lessons provide a design reference, not an already-integrated architecture voice service.

That work needs narration tied to each bundle revision and beat, speech synthesis, measured word/cue alignment, and an audio-driven map to the packet/preview clock. Pause, forward seek and rewind must retain their meaning. Inspecting a function should pause and checkpoint the parent recording; returning should restore its position without late or overlapping audio. Cancellation, missing credentials and unavailable speech must have clear states.

> Release status. Architecture voice synthesis, measured speech synchronization, voice-aware drill-down and restoration are planned, not implemented in this snapshot. Visual-lesson narration capabilities described next do not apply automatically to architecture walkthroughs.

06 / NARRATION SYNCHRONIZATION

Align meaning to the recorded words

This is the visual-lesson/native-hybrid path, not architecture narration. Writing narration and synthesizing speech are separate stages. The recording is aligned to measured word timestamps; exact spoken phrases anchor visual actions. The timeline follows that recording instead of estimating reveals from text length. [1,2]

Native actions now use the same principle

New native scenes declare setup, operation, and result cues. Compilation records their actual runtime events. The alignment request receives those events and their phase meanings, then maps each to a distinct phrase in the existing narration. Missing, inverted, or excessively compressed native action anchors stop publication instead of silently falling back to linear stretching.

The player uses a piecewise audio-to-scene map. It holds the previous completed diagram through the incoming spoken bridge, introduces the new construction at its visual handoff, and preserves time to read the result. Pause, arbitrary forward seek, and rewind reconstruct the same scene time from the same audio position.

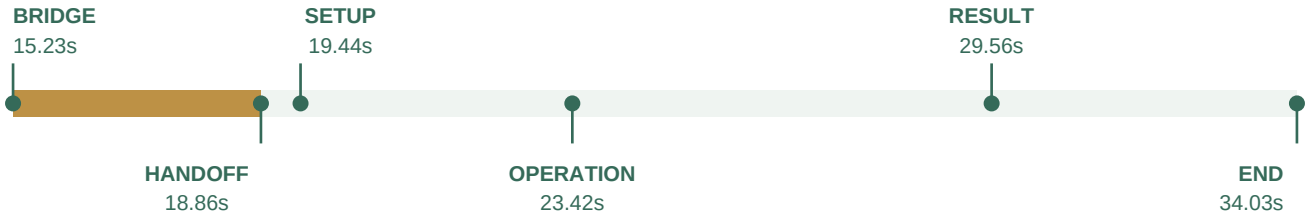


Figure 3. Actual integration fixture, second beat. Narration enters the chapter at 15.23 seconds, but the previous diagram remains until the bridge ends at 18.86 seconds. These are measured times from one recording, not timing targets for other lessons.

Observed event	Audio time, seconds
Second chapter / bridge begins	15.23
New native construction takes over	18.86
Starting vectors introduced	19.44
Scaling operation anchor	23.42
Result anchor	29.56
Recording ends	34.031

Repair anchors without replacing the recording

One bounded anchor correction can reuse the original recording and measured words. Cue-policy changes invalidate stale mapping checkpoints. Existing saved lessons remain loadable through the legacy path, but a cue-less native lesson needs regeneration to acquire the new semantic checks.

The unpublished two-beat check used generated source, Fish audio and Whisper alignment, with browser pause/seek/rewind verification. It does not approve the original 20-beat lesson.

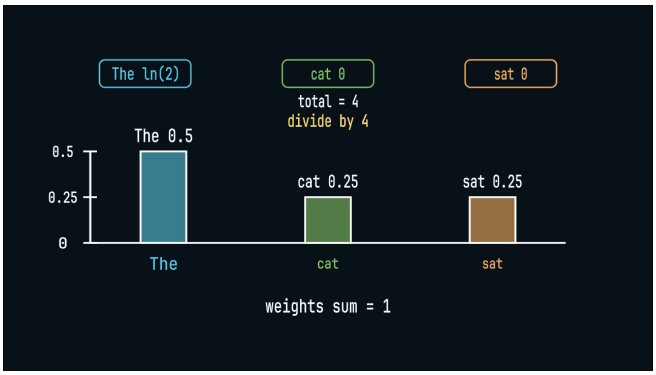
07 / NATIVE QUALITY CONTROLS

Compilation is no longer approval

The inspected Transformer failure case generated 20 native beats in 47.8 seconds. The hybrid player used only seven, each about 2.0-2.4 seconds long. None had semantic runtime cues. Short-video budgets, unused generated chapters, and compilation treated as approval undermined the explanation. The original saved lesson is preserved and needs regeneration. [2]

Harness change	What it now checks or requires
Lesson-specific prompt	Removes the whole-lesson 30-50 second quota; budgets teaching time per native beat from its explanation.
Visible mechanism	Requires a worked construction with inputs, meaningful change and a result; retains the example, source constraints and safeguards.
Runtime contract	Validates cue coverage, ordering, reading time, separation between phases, and a settled result hold.
Readability	Checks overlap and camera-projected text placement, including a clear lower region for player captions.
Teaching gates	Requires source/factual and narrative checks plus actual native and browser frames before publication.

Sampled native results from the integration fixture



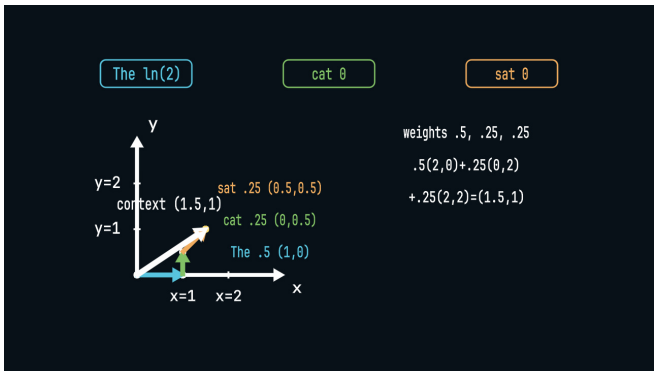


Figure 4. Model-generated native frames, without hand-edited Python: normalization gives 0.5, 0.25, 0.25; weighted value vectors sum to (1.5, 1). These selected settled frames illustrate the integration check. They do not prove that every intermediate state or future lesson is pedagogically sound.

Frame capture uses the playback native host and the final measured timeline. Receipts identify source and frame hashes and the expected renderer revision, preventing a stale image from serving as evidence for changed code. Semantic review remains separate from compilation and mechanical auditing.

Valid CE arrow and axis tip options previously rejected by the compatibility layer are now supported. Repeated numeric labels in separate cards are no longer mistaken for duplicate titles. These repairs improve execution and review accuracy; they do not excuse incorrect numerical geometry or unexplained mechanisms.

08 / GENERATION PERFORMANCE

Most time was spent before playback

The inspected native Transformer run took 590.251 seconds, about 9 minutes 50 seconds, without TTS. Completed phase receipts sum to 587.433 seconds; polling and other overhead explain the small difference. Model work, retries and provider variation dominate this example. [2]



Storyboard Layout / artwork Native scene Fitting

Completed phase	Wall seconds	Share of phase sum
Storyboard	81.831	13.9%
Layout / artwork direction	283.884	48.3%
Native scene preparation	211.116	35.9%
Narration fitting	10.602	1.8%

Native preparation contained 193.686 seconds of model code generation and 17.396 seconds of compilation/auditing. The call log shows overlap in approximately four-worker waves, so the provider was not fully serializing every artwork request. The layout stage made 17 model calls for 12 directed groups; validation attempts and slow responses mattered.

Changes aimed at the measured waste

Only selected native beats are now generated as Python. The direction request retains each target beat's full presentation contract but omits other beats' repeated presentation rationales. For this failure case, the repeated progression field shrank from 12,901 to 6,611 characters. That is a measured payload reduction, not a measured whole-run speedup.

Native code generation defaults to medium reasoning, with high reasoning for its bounded repair. A small same-fixture probe measured 79.5 seconds of medium code generation versus 157.5 seconds for high; the medium-only output was rejected before publication. Two individual probes are not a dependable performance forecast.

Read the receipts correctly

Generation and native-generation receipts separate completed phase work, source reuse, code generation, compilation and repair. Provider logs now retain returned usage, reasoning/cache details and allowance metadata where supplied. An output ceiling is not actual token usage; overlapping request durations are not additive wall time.

SVG animation and native playback do not require a model call for every displayed frame. Both generation routes consume model quota; speech is a separate service. No universal seven-minute Manim penalty or post-fix 20-beat speedup is established.

09 / MODEL SELECTION

Compare usable results, not response speed alone

The inspected lesson route uses OpenCode Go / Muse Spark 1.3 Contributor. Providers and roles remain configurable independently from the voice. Route names and reasoning settings identify the tested configuration; they are not a guarantee of availability, price or quality. [1,2,5]

A bounded comparison sent the same two-beat native task twice to five Go models: requested low reasoning, a 4,096-token ceiling, temperature 0.2, at most three concurrent calls, no fallback, and no automatic retries. This is a small code-generation probe, not an end-to-end lesson benchmark.

Model	Seconds, two trials	Observed result
Muse 1.3 Contributor	27.5 / 32.6	Complete code both times. Valid CE options needed adapter fixes; unchanged outputs then compiled and passed mechanical audit.
GLM-5.3 Flash	23.5 / 26.8	Faster responses, but both outputs used conflicting wait duration arguments.
Qwen3.8 Flash	50.0 / 38.7	Both outputs supplied invalid two-component positions to move_to.
DeepSeek V4.1 Flash	48.5 / 25.4	Both used the requested cap on reasoning and returned no final code. A larger budget could change this result.
MiMo V2.6 Flash	100.5 / 100.4	Both requests timed out; unknown usage is not zero cost.

Cost and capability boundaries

On the official Go page checked for this report, Muse 1.3 Contributor's listed input/output rates are $0.10 / $0.20 per million tokens, excluding cache discounts. It is among the cheapest paid choices, tied with 1.2 Contributor; temporary free choices also exist. Go allowance accounting and actual returned usage must be reported separately from a simple list-price estimate. [5]

Contributor endpoints have a training-data tradeoff. Meta's guide excludes Contributor from its maximum reasoning configuration, so maximum-effort headline benchmarks do not establish this endpoint's teaching capability. No account privacy or model preference was changed by the comparison. [5,6]

Decision

The evidence supports retaining Muse as a reasonable default while validating every candidate. It does not establish a single fastest, cheapest and most capable model. Mechanical success is insufficient: a compiled comparison candidate still had misleading bar heights and intermediate overlaps. Useful comparisons must count failures, repairs, total latency, returned usage and independently judged teaching quality.

10 / ANDROID DELIVERY

Browser studio and Python in one APK

This describes the earlier visual-lesson Android wrapper. The architecture preview added in this revision uses Node on desktop and has not been integrated or verified in that APK.

The implemented Android architecture uses a native WebView with a Chaquopy-hosted Python server at 127.0.0.1:4174. Relative API, narration and media requests remain on the same origin. This wrapper does not use Capacitor. Python 3.11 and the existing assistant package are bundled, with pypdf for PDF extraction. [1,3]

Build or runtime boundary	Current configuration
Android versions	Minimum API 26 / Android 8.0; target and compile API 35. Android 11 / API 30 is within the declared range.
CPU architectures	arm64-v8a phones and x86_64 emulators. This build does not target 32-bit-only Android devices.
Local functionality	Browser scene generation/playback, lesson APIs, contextual questions and quizzes use the bundled Python backend. Remote services still need credentials and network access.
Desktop dependencies	No bundled desktop OpenGL/Manim runtime, CLI runner, LibreOffice or desktop speech executables. Desktop speech alignment cannot be assumed to work inside the APK.
Data	App-private storage for configuration and credentials; Android backup disabled. Keep keys and signing material out of Git.

Background work has limits

PythonService owns the local server. Active work uses a dataSync foreground notification and a bounded partial wake lock. Idle work releases the lock; an idle service stops after the Activity unbinds. Browser-frame review still requires an active WebView. A foreground service does not guarantee uninterrupted execution: Android and the user can stop it. For apps targeting Android 15 or later, dataSync background time is limited to six hours per 24-hour period; the timeout handler stops the service. [4]

Build and update

```
npm run android:check
npm run android:build
# Output: android/app/build/outputs/apk/debug/app-debug.apk
```

The build needs JDK 17 or 21, the licensed API 35 SDK/build tools, Node/npm and a Python 3.11 build interpreter. Develop UI changes with npm run dev; rebuild and reinstall when source changes should appear in the APK. In-app settings and newly generated lessons do not require rebuilding.

The documented output is a debug APK. The current release build type also references debug signing; distribution requires replacing it with a private release configuration. Android 15 emulator checks are documented; physical ARM64, Android 11, screen-lock/provider jobs and network-loss behavior need device verification. Declared compatibility is not a completed device test.

11 / VERIFICATION AND REPRODUCTION

Keep tests and teaching evidence separate

Evidence class	What was established for this update
Python checks	Architecture/proxy and OpenCode subset: 18 passed for this update. The report 2.0 full suite (518 passed, 2 skipped) and earlier 194-test native subset are historical results, not a new full-suite run.
Frontend checks	130 tests passed, plus host and complete architecture TypeScript checks and the production build. Coverage includes unchanged prompts/schemas, provider transport, compiler fidelity, enrichment, reactive timing and rewind.
Live architecture checks	Tutor and gap-filler requests returned HTTP 200 through Go/Muse. One identity-function fixture took about 5.5 and 9.8 seconds respectively; not a latency benchmark. Browser checks verified logo navigation and the provider label.
Earlier native evidence	The report 2.0 fixture used two native beats, actual Fish audio, Whisper word timing and rendered frames. It does not validate architecture narration.
Provider probe	Ten bounded generation requests across five models. Failures and timeouts retained; no universal model ranking or fresh 20-beat speedup claimed.
Android evidence	Earlier debug APK/signature and Android 15 x86_64 emulator checks cover the bundled server, assets, settings, pypdf import and foreground-service startup. They do not cover this native desktop renderer.

Source workflow

```
python3 -m venv .venv
.venv/bin/python -m pip install -e ".[dev]"
npm ci
npm run build
.venv/bin/python -m assistant.visual.server --port 4174
# Open http://127.0.0.1:4174
```

New generation needs the chosen provider's credentials. New narration needs the configured voice service and its alignment dependencies. Existing recorded narration can be replayed without synthesizing it again. Optional native playback needs the separate ManimGL Python environment and a working graphics context; consult the native-runtime documentation rather than assuming the base Python install contains them. [1]

Preserve reproducible evidence

The native review script captures actual rendered frames. The model comparison script records bounded requests and returned usage. The synchronization smoke script can resume against an existing source and recording. Keep inputs, source hashes, renderer revisions, model IDs, requested budgets, failed candidates and timing receipts with a benchmark.

The integration preview is a local development artifact, outside the published lesson catalog. Its localhost URL and cached model files are not a portable release. A public reproduction needs sanitized inputs, sources, media, receipts and dependency instructions. Do not substitute an authored fixture or cached response for a failed live provider run.

12 / LIMITATIONS AND NEXT WORK

Stronger gates still need stronger evaluation

Automated content and frame review can miss a false claim, an ambiguous diagram, or a mechanism that is asserted rather than shown. Sampled frames can miss a bad transition between samples. The updated harness enforces concrete requirements; it does not make every model output consistently good. Expert review and learner studies remain separate work.

What remains from the earlier layout proposal

Proposed intervention	Still requires implementation or evaluation
Perimeter ports	General connector attachment and clearance rules that avoid unrelated objects without masking scientific geometry.
Relative annotations	Named anchors, minimum gaps, containment and cycle/unsatisfiable-constraint reporting; preserve meaningful explicit coordinates for maps, plots and constructions.
Image-grounded revision	Feed the relevant failed image and crop to the reviser together with stable IDs, time and current scene data. Textual findings alone omit spatial evidence.
Selective telemetry	Classify text obstruction, intentional containment, paths and intermediate motion in a common coordinate system; bounding boxes alone over-report collisions.

Native overlap and subtitle checks are useful local controls, not implementation of that full general constraint solver. A clearer final frame does not establish a correct animation, and removing text is not an acceptable repair when the text carries necessary meaning.

Platform and operational limits

Linux is the primary development host. The Python/browser architecture is portable, but this report does not establish a clean-machine Windows installation or parity of native dependencies across platforms. Android packaging is implemented with the explicit runtime exclusions on page 11; the new Node-backed architecture API is outside that verified package. Provider availability, latency and token budgets vary; checkpoint reuse changes the work performed by a retry.

Evaluation that would justify stronger claims

Compare old and updated harnesses on a fixed, repeated task set spanning geometry, mechanisms, history, algorithm execution and descriptive comparisons. Freeze source inputs, route IDs, reasoning and output limits, fonts, retries and review criteria. Report denominators, failed requests, repair regressions, stage wall times and actual returned usage.

Use blinded human judgments for factual correctness, method fit, continuity and visible evidence. Claims about understanding, transfer or retention require a separate learner study with appropriate comparison conditions. Neither an attractive interface nor an internal 100% quiz state supplies that evidence.

13 / REFERENCES AND MAINTENANCE

A report that can stay current

The editable source is docs/technical-report.md. The build script draws diagrams, embeds selected frames and updates the PDF linked from the README. The matching ZIP is an incomplete architecture-preview snapshot; its manifest records the exact included files.

```
# In a Python environment with ReportLab and Pillow:
python scripts/build_technical_report.py
```

References

[1] Pahignalo, R. QWEEPS source repository, documentation and tests. Use the Git commit containing this report. Key paths: browser_lessons.py, native_author.py, native_review.py, synchronization.py, native_timing.py, LiveLessonPlayer.tsx, docs/android.md. Repository.

[2] QWEEPS. Native lesson harness investigation, 5 November 2026. Accompanying repository file: docs/native-harness-review-2026-11-05.md. Includes failure case, phase timings, task comparison and integration evidence.

[3] Chaquopy. Gradle plugin / Android integration guide. Official documentation. Accessed 6 November 2026; use the project's pinned Gradle configuration when reproducing the APK.

[4] Android Developers. Foreground service timeouts. Official documentation. Accessed 6 November 2026. Applies to the specified service types and target/API conditions.

[5] OpenCode. Go model availability, pricing and privacy controls. Official documentation. Accessed 6 November 2026. Prices and availability are time-dependent snapshots.

[6] Meta. Model guide and Contributor reasoning availability. Official documentation. Checked during the 5 November investigation.

[7] Vaswani et al. Attention Is All You Need, section 3.2. Paper. Reference for the weighted-value operation illustrated in the integration fixture; not a QWEEPS performance evaluation.

[8] QWEEPS. Architecture preview: docs/architecture-remix.md; vendor/architecture-remix/UPSTREAM.json; architecture proxy/provider tests; bench/runs/architecture-remix3-2026-11-07/. Archive file hashes identify the release snapshot.

Attribution and distribution

Author: Rayven Pahignalo. Software record: DOI 10.5281/zenodo.23791785. QWEEPS application code uses PolyForm Noncommercial License 1.0.0; this report and QWEEPS-authored documentation use CC BY-NC 4.0, subject to the exclusions in LICENSING.md and third-party notices. Earlier MIT grants are unaffected. This update does not change the Zenodo deposit or certify that a previously uploaded archive contains the new revision.

Report images are model-generated QWEEPS integration-fixture frames, selected from the 34.031-second native synchronization check. No credentials, private documents or provider prompts containing user secrets are included. Archive media and source receipts separately when needed for full reproduction.